

LAB 2

Agile Backlog Creation & Sprint Simulation in Jira

PROJECT

Public Bus Live Tracking & Crowding Estimator

JIRA PROJECT KEY

PBLT

STUDENT

Mutturaj Angadi

SRN

PES1UG24AM439

ACADEMIC YEAR

2026

Prepared using Jira Software (Company-managed Scrum project)

Table of Contents

Objective	3
Project Overview	4
Jira Project and Epics	5
User Stories	6
User Story Format	7
Backlog Prioritization	8
Story Point Estimation	9
Sprint 1	10
Sprint 1 Execution	11
Sprint 2	12
Sprint 2 Execution	13
Burndown Chart	14
Reflection	15
Conclusion	16

1

Objective

The goal of this lab was to take the requirements identified in Lab 1 for the Public Bus Live Tracking & Crowding Estimator and turn them into a working Agile backlog inside Jira, then simulate two sprints against it. This meant grouping the requirements into Epics, breaking each Epic down into User Stories, prioritizing the backlog, estimating effort with Story Points, and finally running the stories through a Scrum board.

Epics represent the major functional themes of the system. User Stories sit under each Epic and describe a specific piece of value from the end user's point of view, written as *As a ... I want ... So that* Once the backlog was prioritized and pointed, two one-week sprints were created and the stories were moved across the board from To Do to In Progress to Done. Jira's Burndown Chart was then used to see how the remaining work in Sprint 2 changed over the course of the sprint.

2

Project Overview

Project: Public Bus Live Tracking & Crowding Estimator

Jira Key: PBLT

The idea behind the project is to give commuters a clearer picture of where their bus actually is, how long it will take to arrive, and how crowded it is likely to be — while also giving the transit team a way to keep an eye on the health of the fleet. The backlog was organized around three functional areas:

- Live bus tracking and ETA calculation
- Crowding estimation for approaching buses
- Fleet monitoring, alerts, and handling of stale GPS data

Epic Overview

Epic	Purpose
Live Bus Tracking & ETA	Ingest live GPS data and show riders an accurate arrival estimate.
Crowding Estimation	Estimate how full an approaching bus is likely to be.
Fleet Monitoring & Reliability	Give the operations team fleet-wide visibility, alerts, and a way to handle unreliable data.

3

Jira Project and Epics

PBLT was set up as a company-managed Scrum project in Jira. The three Epics below group the backlog into the functional areas described in the previous section, and are visible in the Epic panel on the left of the backlog view.

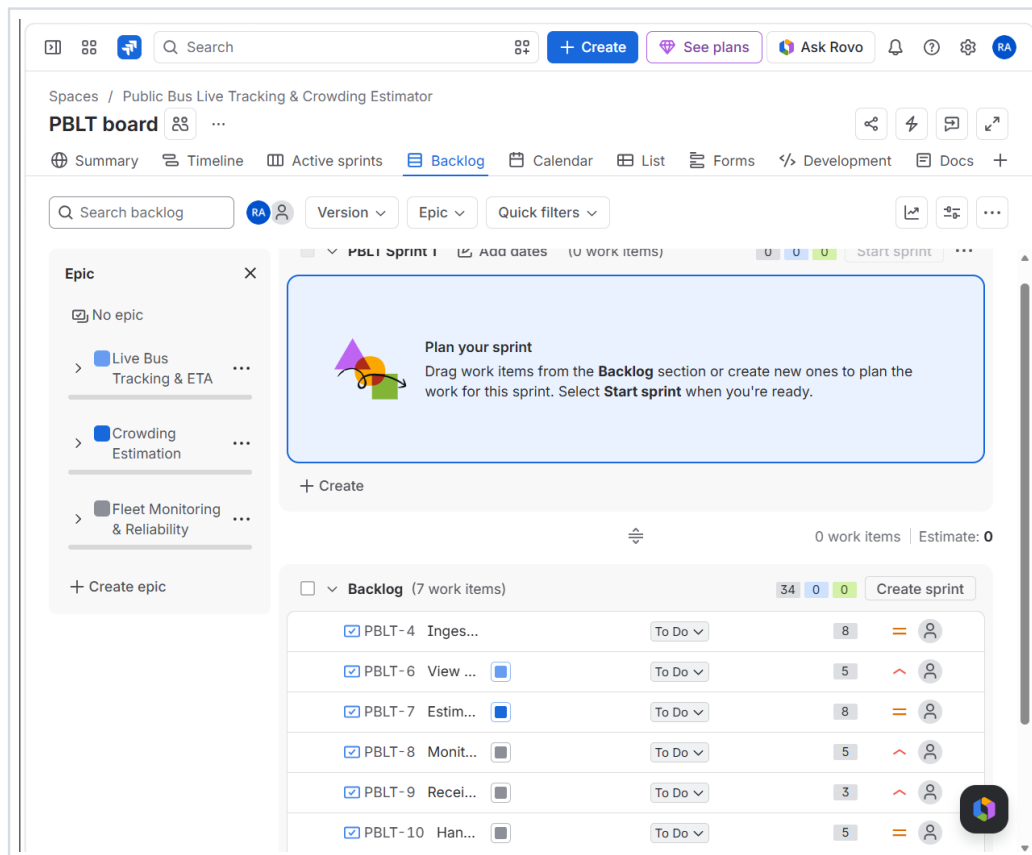


Figure 1. Jira backlog showing the project Epics and associated work items, with the six User Stories and their story points listed below.

- 1. Live Bus Tracking & ETA** — GPS ingestion and the rider-facing ETA screen.
- 2. Crowding Estimation** — Estimating occupancy on approaching buses.
- 3. Fleet Monitoring & Reliability** — Fleet status, alerts, and stale-data handling.

4

User Stories

Each Epic was broken down into User Stories, giving six stories in total across the backlog.

ID	User Story	Epic	Priority	Points
PBLT-4	Ingest GPS and Calculate ETA	Live Bus Tracking & ETA	Medium	8
PBLT-6	View Live Bus ETA	Live Bus Tracking & ETA	High	5
PBLT-7	Estimate Bus Crowding	Crowding Estimation	Medium	8
PBLT-8	Monitor Fleet Status	Fleet Monitoring & Reliability	High	5
PBLT-9	Receive Transit Alerts	Fleet Monitoring & Reliability	High	3
PBLT-10	Handle Stale Data	Fleet Monitoring & Reliability	Medium	5

The two data-heavy stories, Ingest GPS and Calculate ETA and Estimate Bus Crowding, sit under the tracking and crowding Epics respectively, while the three Fleet Monitoring stories round out the operational side of the backlog.

5

User Story Format

Every story in the backlog was written using the standard Agile user-story format:

As a [user role],
I want [goal],
So that [benefit].

This format keeps the focus on the user rather than the implementation — instead of writing "add a GPS parser," the story is framed around what the rider or fleet operator actually gets out of it. Each of the six stories was written this way in its Jira description field, following the same pattern used for the Coffee-Kiosk example in the lab handout, adapted to the bus-tracking domain for each of the three Epics.

6

Backlog Prioritization

Once the stories were written, each one was assigned a priority in Jira based on how directly it affects the rider or the operations team:

- **High** — a feature the rider or operator interacts with directly
- **Medium** — supporting work that a High-priority story depends on

Story	Priority	Rationale
Ingest GPS and Calculate ETA	Medium	Backend work — needed before the ETA can be shown, but not itself user-facing.
View Live Bus ETA	High	The main screen a rider actually looks at.
Estimate Bus Crowding	Medium	Builds on top of GPS tracking, so it naturally comes after it.
Monitor Fleet Status	High	Lets operators catch fleet issues as they happen.
Receive Transit Alerts	High	Time-sensitive — riders need this in real time.
Handle Stale Data	Medium	Reliability work that keeps the other features accurate.

7

Story Point Estimation

Story points were assigned using the Fibonacci scale (0, 1, 2, 3, 5, 8, 13 ...). The gaps between numbers get wider as the values increase, which fits how estimation actually works — it is fairly easy to tell a 2-point task from a 3-point task, but much harder to be precise once a task gets large, so the scale spaces out the higher numbers instead of pretending that level of precision is possible.

The estimates below were arrived at using Planning Poker, discussing each story briefly before settling on a value, as described in the lab handout.

Story	Story Points
Ingest GPS and Calculate ETA	8
View Live Bus ETA	5
Estimate Bus Crowding	8
Monitor Fleet Status	5
Receive Transit Alerts	3
Handle Stale Data	5
Total	34

GPS ingestion and crowding estimation came out as the two heaviest stories, at 8 points each — both involve processing live data and turning it into something reliable, which carries more uncertainty than a simpler feature. Receive Transit Alerts, by comparison, is a fairly self-contained notification feature and was estimated at just 3 points.

8

Sprint 1

Deliver the core real-time tracking, ETA visibility, and crowding-estimation capabilities.

Story	Points
Ingest GPS and Calculate ETA	8
View Live Bus ETA	5
Estimate Bus Crowding	8
Sprint 1 Total	21

Sprint 1 was set up with a one-week duration and scoped to the three stories that make up the foundation of the system — without live GPS data and a working ETA, none of the crowding or fleet features would have anything to build on. This gave the sprint a clear focus: get the core tracking pipeline working end to end before adding the operational features in Sprint 2.

9

Sprint 1 Execution

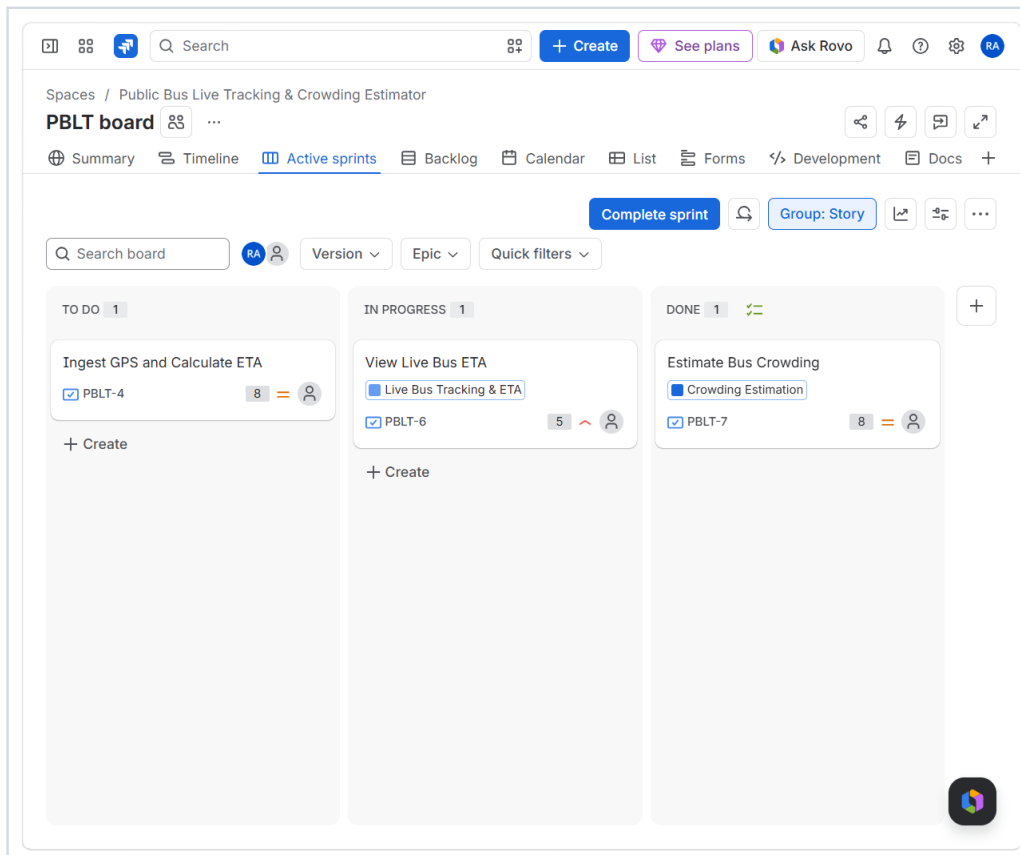


Figure 2. Jira Active Sprint board showing Sprint 1 in progress.

The Scrum board follows the usual three-column workflow — To Do, In Progress, and Done — and each story moves left to right as work on it progresses. In the board above, Ingest GPS and Calculate ETA was still waiting to be picked up, View Live Bus ETA was actively being worked on, and Estimate Bus Crowding had already been completed. As the remaining stories were finished, they were dragged into Done and the sprint was closed out using Jira's Complete sprint action.

10

Sprint 2

Improve operational visibility and reliability through fleet monitoring, alerts, and stale-data handling.

Story	Points
Monitor Fleet Status	5
Receive Transit Alerts	3
Handle Stale Data	5
Sprint 2 Total	13

The screenshot displays the Jira interface for the 'Public Bus Live Tracking & Crowding Estimator' project. The 'PBLT board' is active, showing the 'Backlog' tab. A sprint named 'PBLT Sprint 2' is selected, spanning from 10 Sep to 17 Sep with 3 work items and a total of 13 points. The sprint contains three stories: 'PBLT-8 Monitor Fleet Status' (5 points), 'PBLT-9 Receive Transit Alerts' (3 points), and 'PBLT-10 Handle Stale Data' (5 points). The sidebar on the left shows epic categories: 'No epic', 'Live Bus Tracking & ETA', 'Crowding Estimation', and 'Fleet Monitoring'.

Figure 3. Sprint 2 scoped in the backlog with a one-week duration (10 Sep – 17 Sep) and 13 story points.

With the tracking pipeline in place after Sprint 1, Sprint 2 shifted focus to the operations side of the system — keeping the fleet monitored, alerting riders, and making sure the app degrades gracefully when GPS data goes stale.

11

Sprint 2 Execution

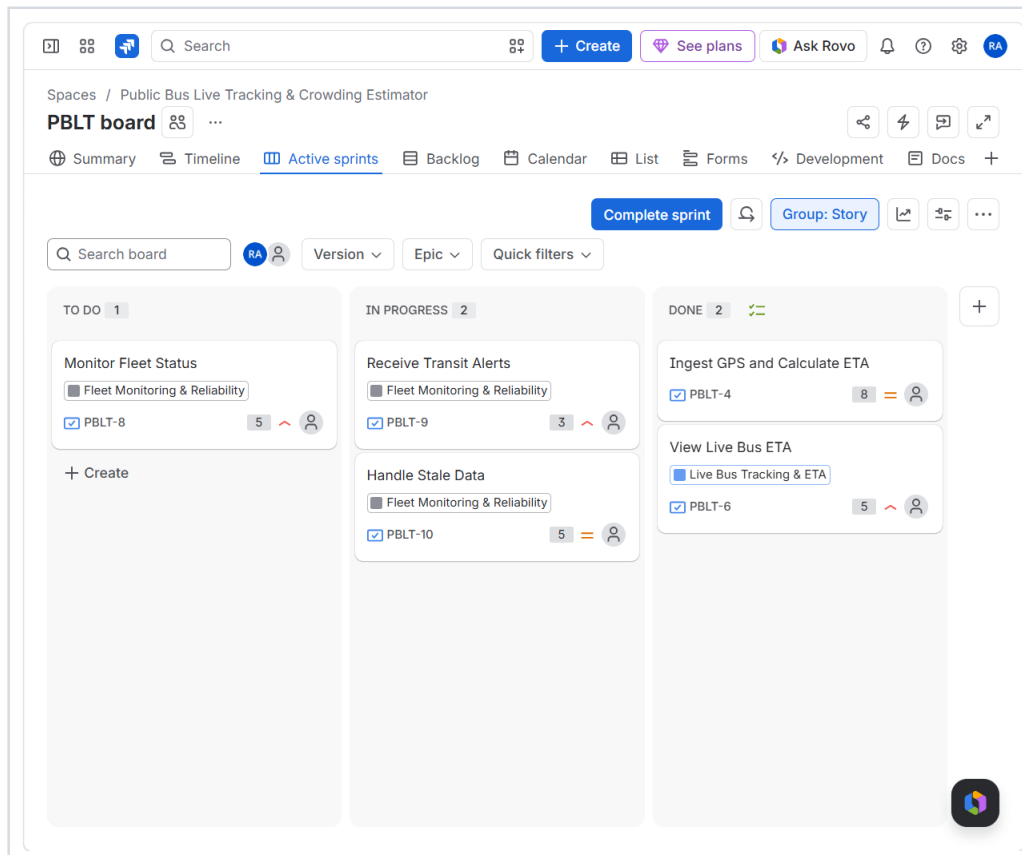


Figure 4. Jira Active Sprint board showing Sprint 2 stories moving through the workflow.

Monitor Fleet Status, Receive Transit Alerts, and Handle Stale Data were carried through the same To Do → In Progress → Done flow used in Sprint 1. All three stories were eventually marked complete and the sprint was closed using Complete sprint. The Burndown Chart on the next page picks up the story from there in more detail, including a small correction that happened right at the end of the sprint.

12

Burndown Chart

The Burndown Chart tracks how many story points are left in a sprint as work gets completed, plotted against a guideline that shows where the team would be if it burned down at a perfectly even rate. It is a quick way to see whether a sprint is on track without having to open the board itself.

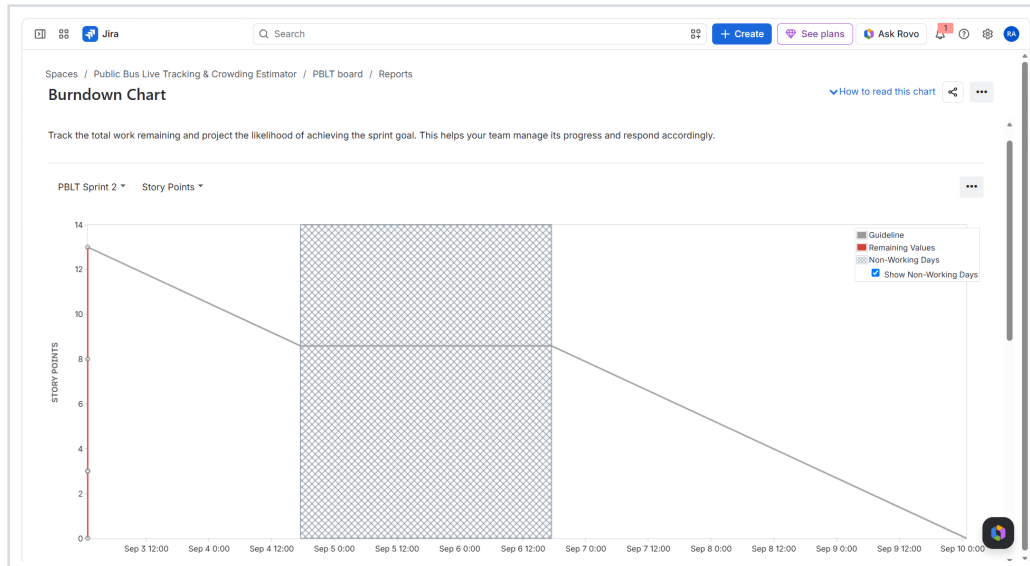


Figure 5. Jira Burndown Chart for Sprint 2, showing remaining Story Points against the guideline.

The figure shows a Jira Burndown event history for Sprint 2. The table lists events, including 'Sprint start', 'Issue completed', and 'Issue reopened', along with the resulting 'Inc.' (Increased) and 'Dec.' (Decreased) story points and the 'Remaining' story points.

Date	Issue	Event Type	Event Detail	Story Points	
				Inc.	Dec. Remaining
03/Sep/26 12:48 AM	PBLT-10	Sprint start		5	13
	PBLT-4			0	8
	PBLT-6			0	3
	PBLT-8			5	0
	PBLT-9			3	3
	Total				13
03/Sep/26 12:49 AM	PBLT-8	Burndown	Issue completed	5	8
	PBLT-10	Burndown	Issue completed	5	3
	PBLT-9	Burndown	Issue completed	3	0
03/Sep/26 12:50 AM	PBLT-9	Burndown	Issue reopened	3	3
	PBLT-10	Sprint ended by RR S&H 2nd Sem P13 MUTTURAJ ANGADI			0
	PBLT-4				0
	PBLT-6				0
	PBLT-8				0
	PBLT-9				3
	Total				3

Figure 6. Burndown event history for Sprint 2, listing each status change and the resulting story points remaining.

Sprint 2 started with all 13 story points remaining. As Monitor Fleet Status, Handle Stale Data, and Receive Transit Alerts were each marked complete, the remaining count dropped in steps — 8, then 3, then 0 — showing the whole sprint burned down. Right before the sprint was closed, Receive Transit Alerts was reopened, which put its 3 points back into the remaining count, so the sprint ended with 3 story points outstanding instead of 0. This is a fairly realistic outcome for a sprint: a story can look done and then get reopened once it is looked at again, and the burndown chart captures that correction rather than hiding it. It is also a reminder to double-check completed work before treating a sprint as fully closed.

13

Reflection

1. Did your estimations reflect the actual effort?

For the most part, yes. The two 8-point stories — GPS ingestion and crowding estimation — were the ones dealing with live data processing, which is genuinely more involved than a simple UI feature, so rating them higher than the rest made sense. Receive Transit Alerts at 3 points turned out to be the one story that needed a second look after being marked done, which suggests its effort wasn't quite as small as the estimate implied, at least in terms of getting it fully correct the first time.

2. Was your backlog well-prioritized?

The priorities line up with what riders and operators would actually notice first. View Live Bus ETA, Monitor Fleet Status, and Receive Transit Alerts were all marked High since they're the parts of the system people interact with directly, while the supporting stories underneath them — GPS ingestion, crowding estimation, and stale-data handling — were Medium. That split kept the backlog focused on what to build first without losing track of the groundwork those features depend on.

3. How did your simulated sprint align with your plan?

Sprint 1's 21 points went toward building the tracking pipeline, and Sprint 2's 13 points went toward the fleet and alerting features on top of it — which is exactly the order planned when the backlog was first prioritized. Sprint 2 mostly went as expected too, though Receive Transit Alerts being reopened near the end meant the sprint didn't close out at a clean zero remaining, which wasn't part of the original plan but is a normal part of how sprints actually play out.

4. What insights did the burndown chart give about your team's capacity?

The chart showed all 13 points burning down to zero fairly quickly once work started, which on its own would suggest the sprint was comfortably within capacity. The reopened story afterward is the more useful insight, though — it shows that a burndown reaching zero isn't the same as a sprint being truly finished, and that some buffer is worth keeping in mind for re-checking work rather than assuming the first Done status is final.

14

Conclusion

This lab gave a hands-on look at how Jira supports the Agile workflow from start to finish. The requirements for the Public Bus Live Tracking & Crowding Estimator were split into three Epics and six User Stories, prioritized by how directly they affect riders and operators, and estimated with Fibonacci story points that added up to 34. Two one-week sprints were then run against that backlog — 21 points in Sprint 1 to build the core tracking and crowding features, and 13 points in Sprint 2 to add fleet monitoring and alerting on top of it. Moving stories across the Scrum board made the idea of sprint progress concrete rather than abstract, and the Burndown Chart for Sprint 2 added a practical lesson beyond the theory: a reopened story near the end of the sprint meant the burndown didn't finish at zero, which is a useful reminder that tracking progress is only half the job — validating that work is genuinely complete matters just as much. Overall, the exercise was a solid, practical introduction to backlog management and sprint execution using a real Agile tool.